

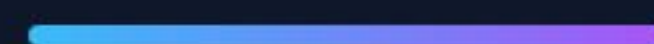
Java Core Engineering </>

البرمجة كائنية التوجه في Java

دليل شامل ومفصّل لمفاهيم الـ OOP الأربعة مع أمثلة برمجية ورسومات توضيحية للذاكرة

التحول البرمجي

من الإجرائية إلى كائنية التوجه



الانتقال من تنظيم الكود في دوال متتالية (Procedural) إلى تنظيم البيانات والسلوكيات في هيئة كائنات واقعية حية (OOP)

الكلاس والأوبجيكت (Class & Object)

1. الكلاس (Class)

هو القالب أو المخطط الهندسي (Blueprint) الذي يحدد الصفات (Attributes) والأنشطة (Methods) قبل إنشاء العناصر.

```
public class Car { String brand; int speed; void  
accelerate() { speed += 10; } }
```

2. الأوبجيكت (Object)

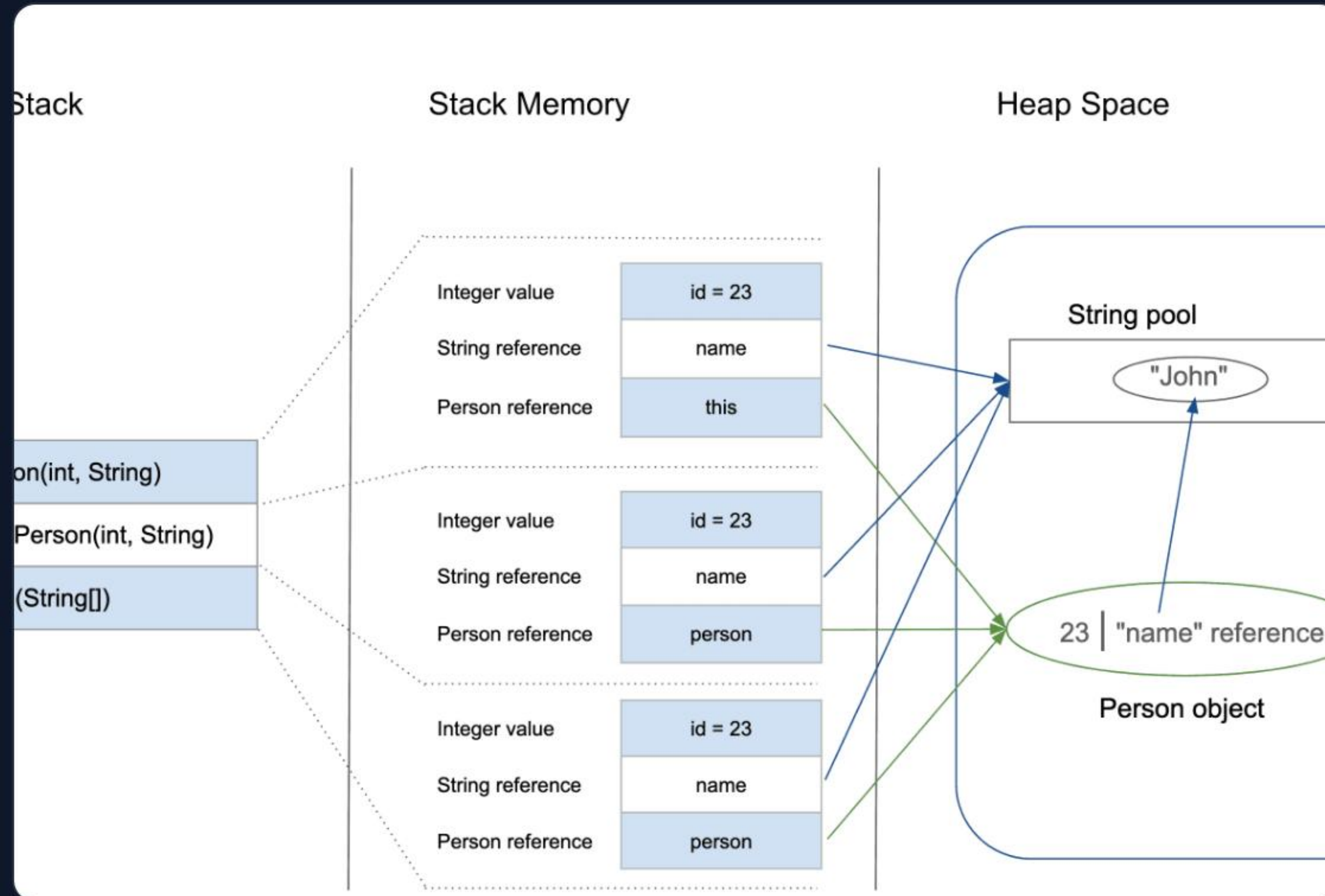
هو النسخة الفعلية (Instance) التي يتم إنشاؤها في الذاكرة باستخدام الكلمة المفتاحية new للحصول على كائن مستقل.

```
public class Main { public static void main(String[] args)  
{ Car myCar = new Car(); myCar.brand = "BMW";  
myCar.accelerate(); } }
```


توزيع الذاكرة (Stack vs Heap)

كيف تدير Java الكائنات في الذاكرة؟

- **ال Stack Memory:** تُخزن فيها المتغيرات المحلية (Local Variables) والمرجعية (References) مثل اسم الكائن myCar.
- **ال Heap Memory:** يُخزن فيها الكائن الفعلي بكل بياناته وحقوله الهيكلية عند تنفيذ new Car().
- **آلية الربط:** يحتوي المرجع في ال Stack على عنوان الذاكرة (Memory Address) الذي يشير للموقع الفعلي في ال Heap.



المبادئ الأساسية

الأركان الأربعة لـ OOP



Encapsulation • Abstraction • Inheritance • Polymorphism

المبدأ الأول: الكبسلة (Encapsulation)

التطبيق في Java

```
public class BankAccount { private double balance; //  
    Encapsulated field public double getBalance() { return  
    balance; } public void deposit(double amount) { if (amount  
> 0) balance += amount; } }
```

المفهوم والأهمية

حماية البيانات الداخلية للكلاس ومنع التعديل المباشر عليها من الخارج، وإتاحة الوصول لها فقط عبر دالتي (Getters & Setters).

الفوائد: التحكم في صحة البيانات (Data Validation)، زيادة أمان الكود، وإخفاء تفاصيل التخزين عن المستخدم.

المبدأ الثاني: التجريد (Abstraction)

التطبيق في Java

```
interface PaymentGateway { void processPayment(double amount); }
class PayPal implements PaymentGateway { public void processPayment(double amount) { // تنفيذ عملية الدفع عبر PayPal } }
```

المفهوم والأهمية

إظهار الوظائف والميزات الأساسية للكائن فقط، مع إخفاء تفاصيل التنفيذ المعقدة (Implementation Details).

يتم تحقيقه في Java باستخدام **Interfaces** أو **Abstract Classes** لتحديد عقد (Contract) للتنفيذ.

المبدأ الثالث: الوراثة (Inheritance)

التطبيق في Java

```
class Employee { double salary = 5000; } class Developer  
extends Employee { String bonus = "1000"; // يمتلك خيار الوصول للـ salary تلقائياً }
```

المفهوم والأهمية

السماح لكلاس جديد (Child/Subclass) بوراثة الخصائص والدوال من كلاس حالي (Parent/Superclass) باستخدام الكلمة المفتاحية `extends`.

الفوائد: إعادة استخدام الكود (Code Reusability) وتقليل التكرار وبناء هياكل برمجية منظمة.

المبدأ الرابع: تعدد الأشكال (Polymorphism)

التطبيق في Java (Overriding)

```
class Animal { void makeSound() { System.out.println("Some sound"); } } class Dog extends Animal { @Override void makeSound() { System.out.println("Woof Woof"); } }
```

الأنواع الرئيسية

- **Compile-time (Overloading)**: تكرار اسم الدالة مع تغيير عدد أو أنواع البرامترات في نفس الكلاس.
- **Runtime (Overriding)**: إعادة كتابة دالة الابن لتغيير سلوك دالة الأب باستخدام `@Override`.

مقارنة: Abstract Class vs Interface

وجه المقارنة	Abstract Class	Interface
الوراثة المتعددة	لا تدعم الوراثة المتعددة (Single Inheritance).	تدعم التطبيق المتعدد (Multiple Inheritance).
أنواع المتغيرات	تكتفي بالمتغيرات العادية، static، و final.	جميع المتغيرات تكون تلقائياً public static final.
تطبيق الدوال (Methods)	تحتوي على دوال مجردة ودوال مكتملة التنفيذ.	دوال مجردة، ويمكن إضافة default أو static.
محددات الوصول	تسمح بـ public, protected, private.	جميع العناصر تكون تلقائياً public.

أثر الـ OOP في جودة البرمجيات

لماذا نستخدم الـ OOP في المشاريع الضخمة؟

تطبيق مفاهيم OOP بالشكل الصحيح يمنح النظام مرونة فائقة للتوسع (Scalability) وسهولة في اختبار الكود (Testability).

كما يمنع تكرار الكود المكتوب ويحقق مبادئ **SOLID Principles** لبناء معمارية برمجية مستقرة وقابلة للتطوير المستمر.

80%

تقليل تكلفة الصيانة والتطوير

أسئلة ومناقشة (Q&A)

شكراً لوقتكم! يسعدنا الإجابة على جميع استفساراتكم حول Java و OOP.

Software Engineering Team | java-oop-mastery@dev.org

Image Sources

<https://www.baeldung.com/wp-content/uploads/2018/07/java-heap-stack-diagram.png>

Source: www.baeldung.com

